

Controllers in ASP.NET Core Web API

What are Controllers?

- In ASP.NET Core Web API, controllers are responsible for **handling HTTP requests** and **returning data** (usually in JSON or XML format) to the client.
- They manage the **flow of data** between the server and the client.
- **Roles of Controller:**
 - **Request Handling:** Controllers receive HTTP requests and send responses.
 - **Data Processing:** They interact with models to process data and implement application logic.
 - **Response Generation:** Controllers decide the type of response sent back to the client, whether it's data, status codes, error messages, etc

Adding Controller Class

- The Controller class in ASP.NET Core Web API must have the “**Controller**” suffix.
- For example, if developer wants to add a controller with the name **Home**, then the name should be **HomeController**.
- Controllers in ASP.NET Core Web API are typically created by inheriting from the **ControllerBase** class.
 - This provides access to several helper methods and properties that are useful for handling HTTP requests and responses.

Add New Scaffolded Item

▲ Installed

▲ Common

API

▸ Blazor

▸ MVC

Razor Pages

Identity

Layout



API Controller - Empty



API Controller with read/write actions



API Controller with actions, using Entity Framework



API with read/write endpoints



API with read/write endpoints, using Entity Framework

API Controller - Empty

by Microsoft
v1.0.0.0

An empty API controller.

Id: ApiControllerEmptyScaffol

Add

Adding Controller Class

- API Controller – Empty:
- This template creates a basic API controller class without any predefined actions.
- It's suitable when developer wants **full control** over defining own actions and routing.
- Developer needs to use this template when it is preferred to manually define all **CRUD operations** and their corresponding **HTTP verbs** (GET, POST, PUT, DELETE).
- It starts with an **empty controller class** that needs to be populated with the desired actions.

Adding Controller Class

- API Controller with Read/Write Actions:
- This template scaffolds a controller with **basic CRUD actions** (Get, Post, Put, Delete).
- It's useful for quickly setting up a controller that supports standard **RESTful operations**.
- It sets up actions for listing **all** resources, retrieving a resource **by ID**, **creating** a new resource, **updating** an existing resource, and **deleting** a resource.

Adding Controller Class

- API Controller with Actions using Entity Framework:
- This template extends the “API Controller with Read/Write Actions” template by integrating with **Entity Framework Core**.
- It includes actions that **interact with a database** using EF Core for data access operations.
- It sets up actions that use EF Core to perform **CRUD operations** on database entities.

Adding Controller Class

- API with Read/Write Endpoints:
- Similar to “API Controller with Read/Write Actions,” this template includes actions for CRUD operations but provides **more flexibility** in customizing endpoint names and HTTP verbs.
- This template should be used when it is needed to have more control over endpoint naming or to include additional actions **beyond basic CRUD operations**.

Adding Controller Class

- API Controller with Read/Write Endpoints using Entity Framework:
- This template Combines the features of “API Controller with Actions using Entity Framework” with the flexibility of customized endpoint names and HTTP verbs.
- Developer needs to choose this template when it is required to have both **database integration** (via EF Core) and **custom endpoint naming**.

Add New Item - WebAPIDemo

Installed

C#

General

ASP.NET Core

Online

Sort by: Default



Search (Ctrl+E)



	Class	C#
	Interface	C#
	Razor Component	C#
	MVC Controller - Empty	C#
	MVC Controller with read/write actions	C#
	API Controller - Empty	C#
	API Controller with read/write actions	C#
	Razor Page - Empty	C#
	Razor View - Empty	C#
	Razor Layout	C#
	Assembly Information File	C#
	Code File	C#
	Machine Learning Model (ML.NET)	C#
	Razor View Start	C#

Type: C#

Empty API Controller Class

Name: EmployeeController.cs

Add

Cancel

EmployeeController.cs

WebAPIDemo

WebAPIDemo.Controllers.EmployeeController

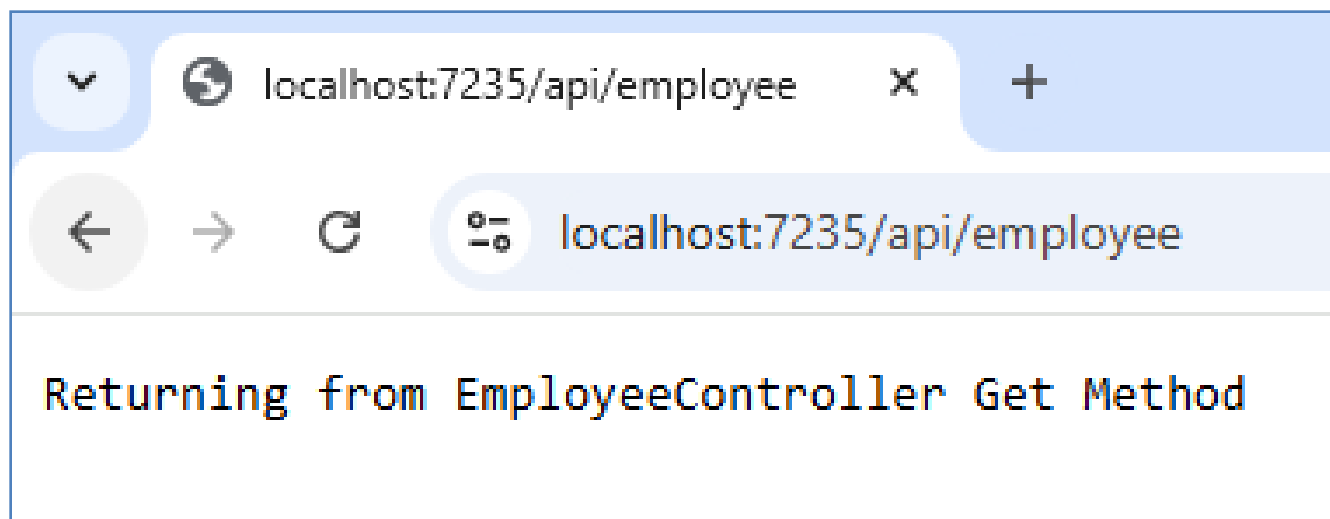
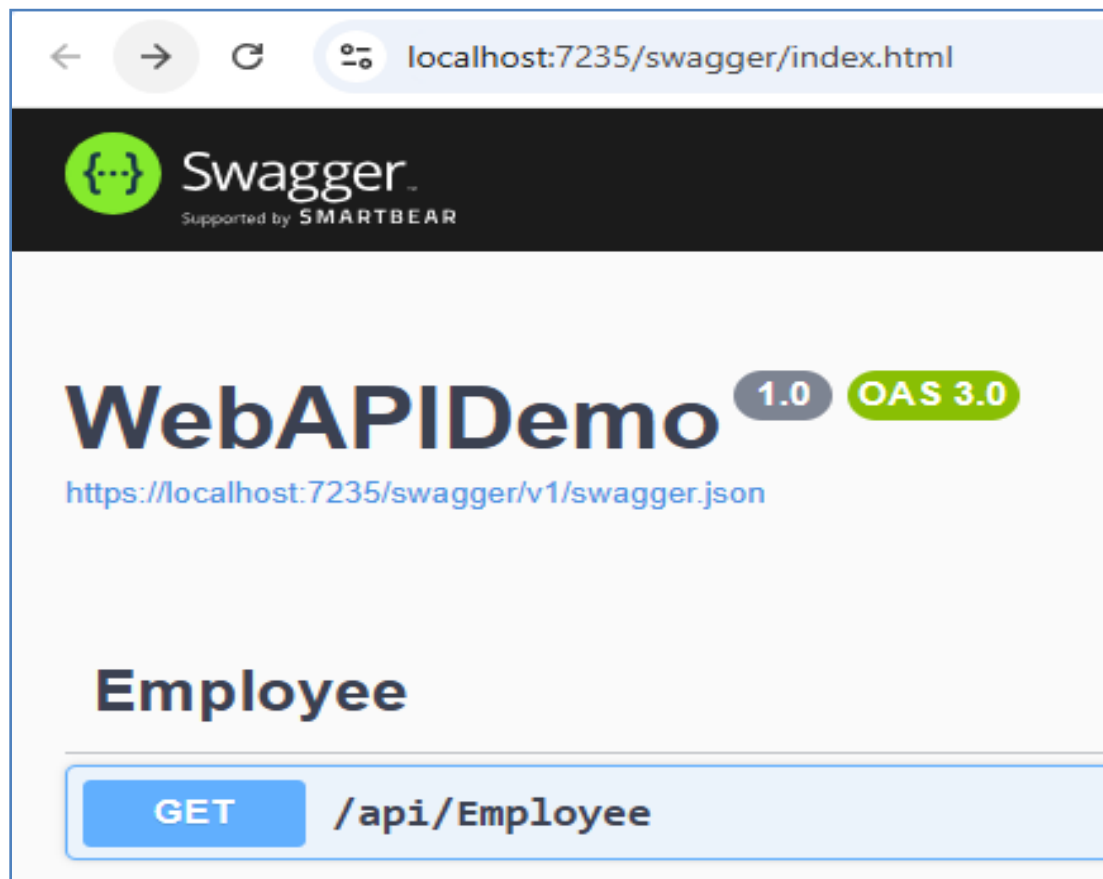
```
1 using Microsoft.AspNetCore.Http;
2 using Microsoft.AspNetCore.Mvc;
3
4 namespace WebAPIDemo.Controllers
5 {
6     [Route("api/[controller]")]
7     [ApiController]
8     public class EmployeeController : ControllerBase
9     {
10     }
11 }
12
```

EmployeeController.cs

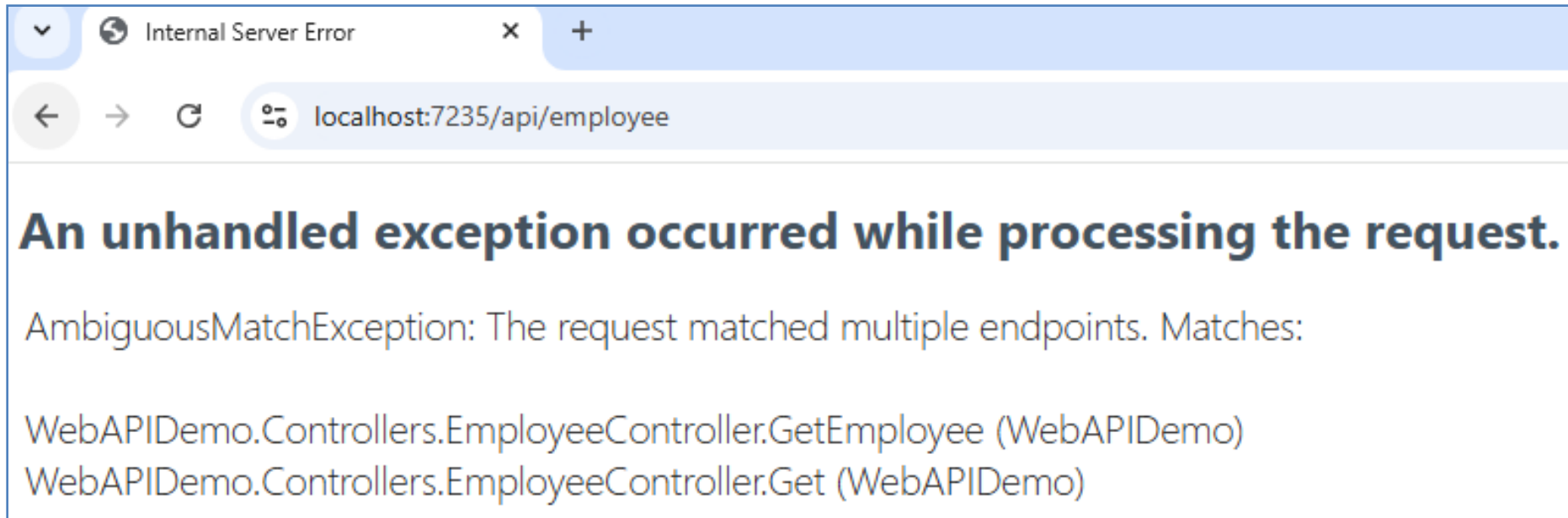
WebAPIDemo

WebAPIDemo.Controllers.EmployeeController

```
1  using Microsoft.AspNetCore.Http;
2  using Microsoft.AspNetCore.Mvc;
3
4  namespace WebAPIDemo.Controllers
5  {
6      [Route("api/[controller]")]
7      [ApiController]
8      public class EmployeeController : ControllerBase
9      {
10         [HttpGet]
11         public string Get()
12         {
13             return "Returning from EmployeeController Get Method";
14         }
15     }
16 }
```



```
1  using Microsoft.AspNetCore.Http;
2  using Microsoft.AspNetCore.Mvc;
3
4  namespace WebAPIDemo.Controllers
5  {
6      [Route("api/[controller]")]
7      [ApiController]
8      public class EmployeeController : ControllerBase
9      {
10         [HttpGet]
11         public string Get()
12         {
13             return "Returning from EmployeeController Get Method";
14         }
15         [HttpGet]
16         public string GetEmployee()
17         {
18             return "Returning from EmployeeController GetEmployee Method";
19         }
20     }
21 }
```



As per Web API standards, each resource should have a Unique Identifier.

EmployeeController.cs* [icon] [X]

[icon] WebAPIDemo

[icon] WebAPIDemo.Controllers.EmployeeController

[icon] Get

```
2  using Microsoft.AspNetCore.Mvc;
3
4  namespace WebAPIDemo.Controllers
5  {
6      [Route("api/[controller]/[action]")]
7      [ApiController]
8      public class EmployeeController : ControllerBase
9      {
10         [HttpGet]
11         public string Get()
12         {
13             return "Returning from EmployeeController Get Method";
14         }
15         [HttpGet]
16         public string GetEmployee()
17         {
18             return "Returning from EmployeeController GetEmployee Method";
19         }
20     }
21 }
```




localhost:7235/swagger/index.html



Swagger

Supported by SMARTBEAR

WebAPIDemo

1.0

OAS 3.0

<https://localhost:7235/swagger/v1/swagger.json>

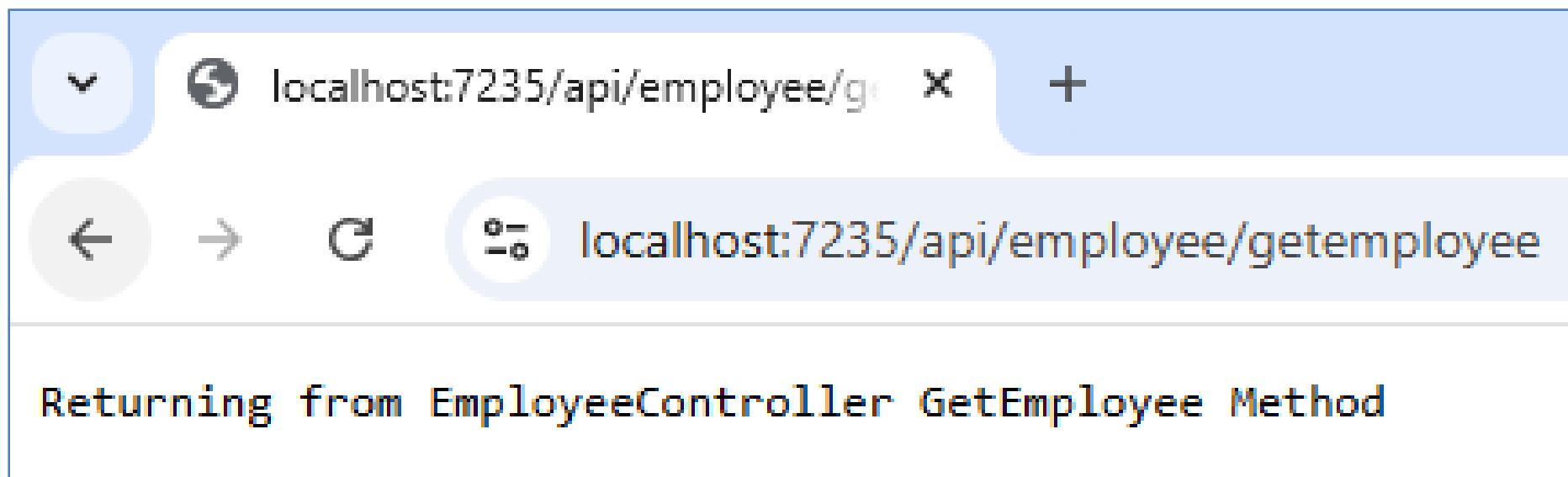
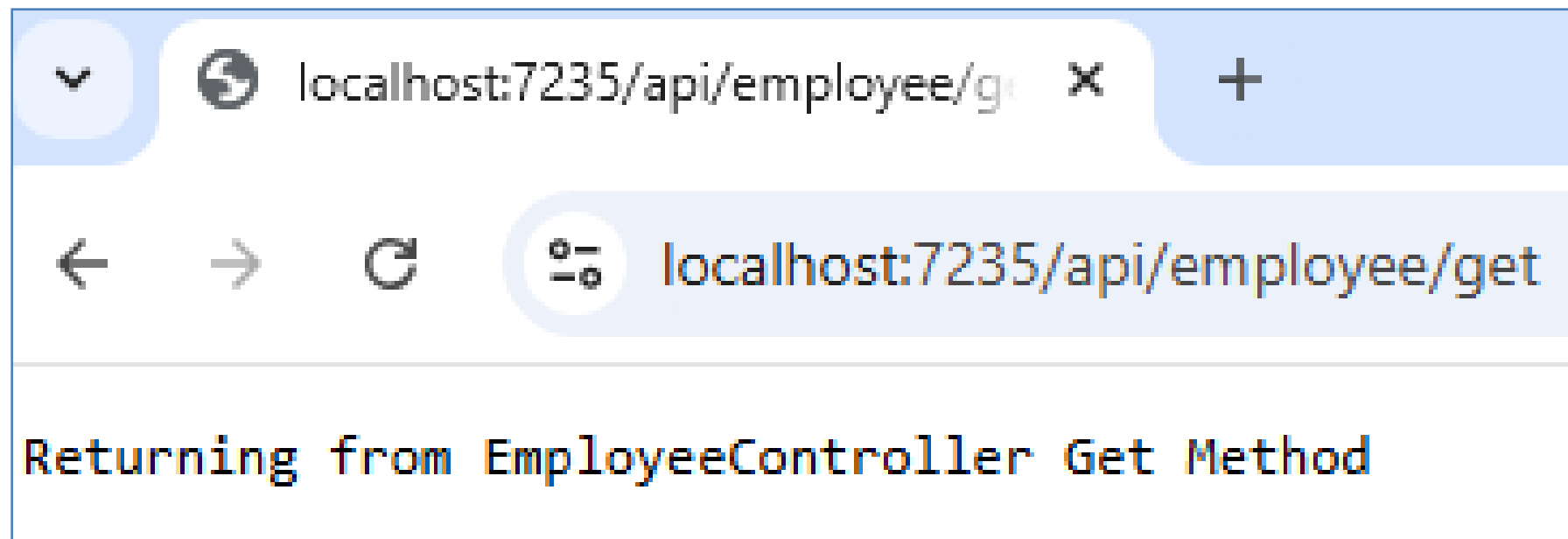
Employee

GET

/api/Employee/Get

GET

/api/Employee/GetEmployee



Features of Controllers

- Controllers use **attributes** to define routes.
- The **[Route]** attribute specifies how to access the controller via URL patterns.
- The **[ApiController]** attribute decorates the class to enable Web API-specific behaviors.
- Inside controllers, methods known as **action methods** respond to HTTP requests.
- Each method is typically decorated with an attribute corresponding to an **HTTP method** ([HttpGet], [HttpPost], etc.), defining what type of request it handles.

Features of Controllers

- Action methods can **accept parameters** directly from the route, query string or body of the request, which ASP.NET Core binds automatically.
- Controllers use **IActionResult** or **ActionResult<T>** to return responses.
- These can encapsulate various **HTTP responses**, allowing for easy customization of status codes and payloads.

Features of ApiController Attribute

- **Automatic HTTP 400 Responses:**
- When applied to a controller, ASP.NET Core automatically validates the model state of incoming HTTP requests.
- **Attribute Routing Requirement:**
- The [ApiController] attribute makes attribute routing mandatory for the controller.
- This means developer must use [Route], [HttpGet], [HttpPost], etc., to define routes for the controller's actions.

Features of ApiController Attribute

- Binding Source Parameter Inference:
- The [ApiController] attribute enhances the binding of incoming data by inferring where to look for data based on the **type of the parameter** and the **HTTP method** of the request.
- For example, in case of a POST method, it assumes binding from the request body.
- Problem Details for HTTP Error Responses:
- When an action method returns an error (e.g., BadRequest, NotFound, Internal Server Error), the [ApiController] attribute automatically generates a ProblemDetails response

Choosing between Controller and ControllerBase Classes

- APIs:
- For building a RESTful API without views, ControllerBase is used to keep the project lightweight and focused on data endpoints.
- MVC Applications:
- If the application requires server-side rendering of views using Razor views, Controller is used to leverage its support for views and related functionalities.